

## UNIT 1

Introduction to

# Laravel

---

Open-source PHP Framework • MVC Architecture • Created by Taylor Otwell (2011)

**2011**

Released

**PHP >= 5.5.9**

Language

**MVC**

Pattern

**Open Source**

License

# Topics Covered in Unit 1

01 · What is Laravel?

02 · Key Features

03 · Core Concepts (MVC)

04 · What Makes Laravel Special?

05 · Installing via Composer

06 · Directory Structure

07 · Your First Project

08 · Routing & Views

09 · Blade Templating Engine

01

# What is Laravel?

*Open-source PHP web application framework*

# Laravel at a Glance

## Open-source PHP Web Application Framework

Free to use, community-driven, huge ecosystem of packages and tools.

## Created by Taylor Otwell in 2011

Quickly became one of the most starred PHP frameworks on GitHub.

## Follows MVC Architectural Pattern

Model-View-Controller separates data, presentation and business logic cleanly.

## Expressive & Developer-Friendly Syntax

Write less boilerplate code; do more with elegant, readable PHP.

## Robust & Scalable

Powers small blogs to enterprise-scale web applications with the same framework.

02

## Key Features

*Why Laravel is a preferred choice for modern web development*

# Laravel Feature Comparison

## MVC Framework

Yes — from Laravel 2 onwards

## Eloquent ORM

Object Relational Mapping for database interaction

## Unit Testing

PHPUnit is built-in to prevent regressions

## Database Migrations

Simplifies deployment & schema updates

## Form Validation

Robust integrated validation methods

## Caching

Built-in caching support to optimize performance

## Security

SSH for CLI, CSRF protection, input sanitization

## Scaffolding

Specify application database usage quickly

## Rapid App Development

Facilitates quick development cycles

## Language Support

PHP Version  $\geq$  5.5.9

03

## Core Concepts

*The fundamental MVC building blocks of every Laravel application*

# MVC Architecture in Laravel

## Routing

routes/ — Maps incoming URLs to controller actions. Defines how the app responds to requests.

## Controllers

app/Http/Controllers/ — Handles requests, processes data, determines responses.

## Models

app/Models/ — Represents data & business logic. Interacts with DB via Eloquent ORM.

## Views

resources/views/ — User interface via Blade templating engine. Clean PHP-in-HTML syntax.

# Reasons to Learn Laravel

## Robust Framework

Eloquent ORM, Blade templating and artisan CLI simplify complex development tasks.

## Active Community

Extensive documentation, thousands of tutorials, Laracasts video platform.

## Scalability

Modular structure and built-in tools support growing applications effortlessly.

## Job Opportunities

High demand for Laravel developers — one of the most hired PHP skills globally.

## Future-Proofing Skills

Build modern, high-performance web applications with confidence.

04

## What Makes Laravel Special?

*First-party packages that give Laravel a competitive edge*

# First-Party Laravel Packages

## Scheduler

Support for scheduling periodic background tasks using cron-like syntax.

## Cashier

Manages subscription billing services (Stripe / Paddle integration).

## Flysystem

Remote storage usage similar to local file systems — S3, FTP, SFTP.

## Socialite

Simplifies OAuth authentication with Facebook, GitHub, Google, etc.

## Sanctum

Lightweight token-based API auth for SPAs and mobile applications.

## Breeze

Minimal, clean authentication starter kit with Blade + Tailwind CSS.

## Horizon

Redis queue monitoring dashboard — monitor jobs, failures & throughput.

## Telescope

Debug assistant showing requests, exceptions, jobs, queries and more.

# Advanced Laravel Capabilities

## Authentication & Authorization

Secure user login, registration and role-based access control (Gates & Policies).

## Database Seeding & Migrations

Manage schema evolution and populate sample data for testing environments.

## Event System

Coordinate complex logic and decouple components using the elegant event/listener pattern.

## Job Queuing

Handle long-running tasks (email sending, file processing) asynchronously.

## REST API Development

Build web services for mobile apps, frontend frameworks and external systems.

05

# Installing Laravel via Composer

*Dependency management, autoloading & version compatibility*

# Why Use Composer for Laravel?

## Dependency Management

Lists required libraries in `composer.json`; resolves & installs all automatically.

## Autoloading

Generates optimized autoloader; no manual `require/include` statements needed.

## Version Compatibility

Specify version constraints per package to avoid conflicts between dependencies.

## Updates & Maintenance

Run ``composer update`` to get latest bug fixes, security patches and features.

## PHP Package Ecosystem

Integrates with Packagist — the main PHP package repository (thousands of packages).

## Download Composer

<https://getcomposer.org/> | PHP: <https://windows.php.net/download/>

# Installing Laravel — Step by Step

1

## Install Composer & PHP

Download from [getcomposer.org](https://getcomposer.org) and [windows.php.net](https://windows.php.net). Verify with ``php -v``.

2

## Create Laravel Project

Run: `composer create-project --prefer-dist laravel/laravel project-name`

3

## Navigate & Configure .env

`cd project-name` → Copy `.env.example` to `.env` and set your settings.

4

## Generate Application Key

Run: `php artisan key:generate` (secures encrypted data)

5

## Start Development Server

Run: `php artisan serve` → Visit `http://localhost:8000`

# Composer Install & Artisan Commands



# Step 1: Create a new Laravel project

```
composer create-project --prefer-dist laravel/laravel project-name
```

# Step 2: Navigate into the project

```
cd project-name
```

# Step 3: Copy environment config

```
cp .env.example .env
```

# Step 4: Generate the application key

```
php artisan key:generate
```

🚀 After 'php artisan serve' open <http://localhost:8000> in your browser to see the welcome page.

```
php artisan serve
```

06

# Laravel Directory Structure

*Understanding each folder and its role in the framework*

# Top-Level Directories

## app/

Heart of the framework — Controllers, Models, Middleware, Providers

## bootstrap/

App bootstrapping; cache directory for performance optimization

## config/

All configuration: database, mail, session, cache, services

## database/

Migrations, model factories, and database seeders

## public/

Entry point index.php + compiled assets (CSS, JS, images)

## resources/

Frontend: Blade templates, raw CSS, JavaScript source files

## routes/

All route definitions: web.php, api.php, console.php

## storage/

Compiled views, file sessions, caches, user-uploaded files

## tests/

Automated tests — Feature and Unit test directories

## vendor/

Composer dependencies (never edit manually; not committed to git)

## Sub-directories Inside app/

| Sub-directory | Purpose                                   | Create With       |
|---------------|-------------------------------------------|-------------------|
| Console       | Custom Artisan commands                   | make:command      |
| Exceptions    | Application exception handling files      | —                 |
| Http          | Controllers, Middleware, Form Requests    | make:controller   |
| Providers     | Service providers — register app services | make:provider     |
| Events        | Event classes for component signaling     | make:event        |
| Jobs          | Queueable job classes                     | make:job          |
| Listeners     | Classes that handle Events                | make:listener     |
| Mail          | Email classes sent by the application     | make:mail         |
| Notifications | Transactional notification classes        | make:notification |

07

# Creating Your First Project

*Route → Controller → View: the complete request cycle*

# Your First Laravel Project — Walkthrough

1

## Install & Create Project

`composer create-project laravel/laravel my-app`

2

## Configure Environment

Edit `.env` → set `DB_DATABASE`, `DB_USERNAME`, `DB_PASSWORD`

3

## Run Migrations

`php artisan migrate` (creates default tables)

4

## Define a Route

`routes/web.php` → `Route::get('/', fn() => view('welcome'))`

5

## Create a Blade View

`resources/views/welcome.blade.php` → `HTML + {{ $var }}`

6

## Start & Test

`php artisan serve` → visit `http://localhost:8000`

08

# Routing & Views

*Defining routes, returning strings, views and using controllers*

# Basic Routes in web.php



```
// routes/web.php
```

```
// Route returning a plain string
```

```
Route::get('/sayhello', function() {  
    return 'Hey! Hello';  
});
```

```
// Route returning a Blade view
```

```
Route::get('/viewhello', function() {  
    return view('index'); // serves resources/views/index.blade.php  
});
```

🚩 *Routes are case-sensitive: /profile ≠ /Profile. All web routes go in routes/web.php.*

```
Route::get('/viewindex2', [MyController::class, 'index']);
```

# Creating & Using a Controller



```
// app/Http/Controllers/MyController.php
<?php
namespace App\Http\Controllers;
use Illuminate\Http\Request;

class MyController extends Controller {
    public function index() {
        return view('index2');
    }
}
```

```
// Syntax: Route::get('/url', [ControllerName::class, 'methodName']);
```

🚀 *Generate controllers quickly: php artisan make:controller MyController*

09

# Blade Templating Engine

*Views, data binding, composing, sharing and view creators*

# Blade View Methods

## **View::exists('name')**

Returns true/false — check if a view file exists before rendering.

## **view()->first([...])**

Render the first view found from an array of view names.

## **{{ \$variable }}**

Escaped output via htmlspecialchars — protects against XSS attacks.

## **{!! \$variable !!}**

Unescaped raw HTML output — use only with trusted content.

## **view('name')->with('key','val')**

Pass specific data to a single view as named variable.

## **View::share('key','value')**

Share a variable with ALL views in the application globally.

## **View::composer('view', fn)**

Bind data to a view whenever it is rendered (lazy).

## **View::creator('view', fn)**

Bind data immediately when the view object is created (eager).

# Data Binding & XSS Protection



```
// Passing data to a view (routes/web.php)

Route::get('/', function () {
    return view('profile')->with('name', 'Aakash')->with('age', 21);
});
```

```
// resources/views/profile.blade.php

<h3>My name is {{ $name }} and my age is {{ $age }}</h3>

<!-- {{ }} → escaped (safe, use for user input) -->

<h3>Unescaped: {!! $htmlContent !!}</h3>

<!-- {!! !!} → unescaped (only for trusted HTML) -->
```

```
// Global change in AppServiceProvider::boot()
```

✦ Always use {{ }} for user-provided data — Blade auto-escapes to prevent XSS attacks.

# View Composer vs View Creator

## View Composer — Lazy Binding

```
View::composer('shared', function($view) { $view->with('data', 'value'); })
```

Executes when the view is about to render. Ideal for dashboard widgets, navigation menus, etc.

## View Creator — Eager Binding

```
View::creator('shared', function($view) { $view->with('data', 'value'); })
```

Executes immediately when the view object is instantiated (before rendering).

## Sharing with Multiple Views

Pass an array: `View::composer(['shared','sidebar'], fn) — or use '*' wildcard for all views.`

## Where to Register

Add to the `boot()` method of `app/Providers/AppServiceProvider.php` (loaded on every request).

# Unit 1 Summary

## *Introduction to Laravel — Key Takeaways*

---

### Laravel

is an open-source PHP MVC framework created by Taylor Otwell in 2011

### Blade

is Laravel's templating engine; use `{{ }}` for safe output, `{!! !!}` for raw HTML

### Composer

manages all dependencies; install with ``composer create-project laravel/laravel``

### Directory

`app/` is the core; `routes/` for routes; `resources/views/` for templates

### MVC

Routes → Controllers → Models → Views — the backbone of every Laravel app

### Artisan

``php artisan serve`` starts dev server; artisan has 100+ helper commands